

Una (Brevissima) Introduzione al Machine Learning

Metodi Statistici e Matematici per il Machine Learning

Traduzione italiana — materiale originale a cura di Davide Evangelista

Nelle prossime sezioni studieremo la **Discesa del Gradiente** (*Gradient Descent*, probabilmente il metodo di ottimizzazione più conosciuto e utilizzato) e la sua variante stocastica, la **Discesa del Gradiente Stocastica** (*Stochastic Gradient Descent*, SGD). Per motivare l'introduzione di questi metodi nel contesto del Machine Learning, iniziamo presentando alcuni concetti fondamentali di Machine Learning da un punto di vista matematico. In particolare, discuteremo come il problema dell'apprendimento possa essere formulato matematicamente, e delineeremo la tipica pipeline usata per comprendere, progettare e risolvere un problema di Machine Learning.

Iniziamo con una definizione ampiamente accettata di **Machine Learning**:

Il **Machine Learning (ML)** è l'insieme di tecniche e algoritmi che estraggono conoscenza dai dati e usano quella conoscenza per fare predizioni accurate.

1 Il problema dell'apprendimento

Un problema di apprendimento consiste nell'estrarre conoscenza da un dataset di N punti:

$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(N)}, y^{(N)})\}$$

dove ogni coppia $(x^{(i)}, y^{(i)})$ è composta da un ingresso (*input*) e dalla corrispondente uscita (*output*). Pensiamo al dato di ingresso $x^{(i)} \in \mathbb{R}^d$ come all'informazione disponibile, e al dato di uscita $y^{(i)} \in \mathbb{R}^s$ come alla quantità che vogliamo predire. La dimensione di ingresso d rappresenta il numero di feature (caratteristiche) in ingresso, mentre la dimensione di uscita s corrisponde al numero di variabili che vogliamo predire. Per semplicità, questi dati vengono solitamente rappresentati come vettori di dimensione d e s rispettivamente.

Ad esempio, il Machine Learning è ampiamente usato per predire il prezzo di una casa a partire dalle sue caratteristiche. Supponiamo di avere un dataset in cui ogni casa è descritta da feature come:

- x_1 : metri quadri
- x_2 : numero di camere da letto
- x_3 : numero di bagni
- x_4 : distanza dal centro città
- x_5 : età della casa

L'obiettivo è predire il prezzo di vendita y di una casa date le sue caratteristiche. Qui, l'ingresso $x^{(i)} \in \mathbb{R}^5$ rappresenta un vettore di attributi della proprietà, mentre l'uscita $y^{(i)} \in \mathbb{R}$ è il prezzo della casa.

Un modello di Machine Learning, come la **regressione lineare**, apprende una funzione f_Θ che mappa gli ingressi nelle uscite:

$$y = f_\Theta(x_1, x_2, x_3, x_4, x_5).$$

Inizialmente, il modello **non sa** come queste feature influenzino il prezzo. Allenandosi (*training*) su dati di vendite passate, trova pattern e stima parametri (o pesi) che meglio si adattano ai dati. Una volta allenato, il modello può predire il prezzo di case nuove, non presenti nel dataset.

Questo semplice esempio illustra l'idea centrale dell'apprendimento: usare dati passati per fare predizioni future. Più in generale, un modello di Machine Learning può essere visto come una funzione matematica che mappa feature in ingresso in un'uscita di interesse.

Attenzione

Questa impostazione assume che tutti i dati siano rappresentati come numeri reali (dato che sia $x^{(i)}$ che $y^{(i)}$ sono vettori a valori reali). Gestire dati non numerici richiede un passo di pre-processing chiamato *embedding*, che verrà discusso più avanti.

1.1 Formulazione matematica dei modelli di Machine Learning

Nella sua essenza, un modello di Machine Learning (ML) è una funzione

$$f_{\Theta} : \mathbb{R}^d \rightarrow \mathbb{R}^s$$

che mappa un vettore di ingresso di lunghezza d in un vettore di uscita di lunghezza s . La scelta di s dipende dall'applicazione:

- Se il modello è usato per **classificazione**, dove un ingresso appartiene a una tra k categorie, allora $s = k$.
- Se il modello è usato per **regressione**, s corrisponde al numero di valori predetti.
- Se il modello è usato per **ricostruzione di immagini**, s rappresenta tipicamente il numero di pixel nell'immagine ricostruita.

La conoscenza del modello è codificata nel vettore Θ , che rappresenta i suoi **parametri**. Per capire meglio il ruolo dei parametri in un modello ML, partiamo da un esempio semplice: la Regressione Lineare.

Consideriamo un modello base f_{Θ} con solo due parametri, Θ_1 e Θ_2 :

$$f_{\Theta}(x) = \Theta_1 + \Theta_2 x.$$

Questa equazione descrive una retta, dove Θ_1 è l'intercetta e Θ_2 è la pendenza.

```
import numpy as np
import matplotlib.pyplot as plt

# Definisce la funzione parametrizzata f
def f(theta, x):
    return theta[0] + theta[1] * x

# Sceglie due diversi valori dei parametri
theta = (1, 0.2)
theta2 = (0, -1)

# Visualizza la predizione del modello nell'intervallo [-5, 5]
xx = np.linspace(-5, 5, 10)
yy = f(theta, xx)
yy2 = f(theta2, xx)

plt.plot(xx, yy, 'o--')
plt.plot(xx, yy2, 'o--')
plt.grid()
plt.xlabel("x")
plt.ylabel("y")
plt.title("Grafico di f_theta(x)")
plt.show()
```

Pensiamo a x come al dato in ingresso che usiamo per predire un'uscita y . Si noti come scelte diverse di Θ portino a predizioni molto diverse tra loro.

Ora sovrapponiamo i dati disponibili sullo stesso grafico: definendo dei punti sintetici $y^{(i)} = 1 + 0.2x^{(i)} + \varepsilon^{(i)}$ con rumore gaussiano $\varepsilon^{(i)} \sim \mathcal{N}(0, 0.5^2)$, e confrontando le due rette candidate con i punti dati, ci si può chiedere: *quale delle due rette si adatta meglio ai dati?* Chiaramente la retta con $\Theta = (1, 0.2)$: infatti, controllando il codice usato per generare i dati, si vede che $(1, 0.2)$ sono esattamente i parametri usati per definire $y^{(i)}$.

Questo mette in luce due caratteristiche fondamentali comuni a quasi tutti i modelli ML:

- Scelte diverse di Θ portano a predizioni *molto diverse*.
- Dato un training set, alcune scelte dei parametri sono migliori di altre, e solitamente ne esiste una *ottimale*. Un buon modello dovrebbe almeno approssimare questa scelta ottimale.

Il processo di ricerca dei parametri ottimali per un modello ML, basato su un training set, è chiamato *training* (allenamento). Torneremo su questo argomento più avanti, quando discuteremo come i modelli vengono tipicamente allenati.

1.2 Linearità

Nell'esempio precedente abbiamo assunto che sia la dimensione di ingresso che quella di uscita fossero $d = 1$ e $s = 1$. Tuttavia, possiamo facilmente generalizzare un modello di regressione lineare a dimensioni maggiori ($d > 1$, $s > 1$) usando il modello lineare generale:

$$f_{\Theta}(x) = Wx + b$$

dove i parametri $\Theta = \{W, b\}$ hanno dimensioni $W \in \mathbb{R}^{s \times d}$ e $b \in \mathbb{R}^s$. Questo significa che il numero totale di parametri da apprendere è $s(d + 1)$, che cresce linearmente con le dimensioni di ingresso e di uscita.

Quando $d > 1$ e $s > 1$, non possiamo più visualizzare l'uscita del modello con un semplice grafico, dato che richiederebbe almeno quattro dimensioni.

Sebbene i modelli lineari abbiano diverse proprietà utili, la loro **espressività** (cioè la capacità di approssimare uscite complesse) è piuttosto limitata. Un modello lineare può rappresentare solo *funzioni lineari* (come rette e piani), rendendolo incapace di approssimare anche funzioni semplici come $\sin(x)$. Questa limitazione rende i modelli lineari poco pratici per gestire dati complessi, come immagini o linguaggio naturale — ad esempio, un modello lineare che tenta di approssimare $\sin(x)$ su $[-5, 5]$ produce una retta che si discosta enormemente dall'andamento oscillante della sinusoide.

Per questo motivo, in letteratura sono stati proposti diversi algoritmi ML, tra cui i più diffusi sono la Regressione Polinomiale, le Support Vector Machine (SVM), le Random Forest (RF) e XGBoost.

2 La pipeline del Machine Learning

L'esempio precedente mostra che progettare un algoritmo di Machine Learning efficace richiede diverse scelte accurate, come la struttura del modello, la quantità e qualità dei dati, e i valori dei parametri. In particolare, possiamo identificare i seguenti passi, comuni alla maggior parte degli algoritmi ML:

- **Comprendere** il compito (ad es. di cosa abbiamo bisogno? quali informazioni possiamo raccogliere per rispondere alla domanda che ci poniamo?),
- **Raccogliere** un dataset sufficientemente ampio, contenente abbastanza informazione rilevante per affrontare il compito,
- **Progettare** l'algoritmo di Machine Learning, sfruttando la conoscenza pregressa sul problema in esame,
- **Allenare** l'algoritmo sui dati raccolti, minimizzando l'errore di predizione sul training set,
- **Calibrare (tuning)** i parametri del modello (in ML, un algoritmo è spesso chiamato *modello*) per migliorarne le predizioni,
- **Testare** l'algoritmo su dati nuovi, per verificarne la capacità predittiva.

Esamineremo ciascuno di questi passi più in dettaglio nelle sezioni seguenti.

2.1 Comprendere il compito

Assumiamo di voler risolvere un dato problema. Matematicamente, il problema che vogliamo risolvere può essere modellato come una funzione (incognita) $f(x)$, che prende in ingresso il vettore $x \in \mathbb{R}^d$ e lo mappa (possibilmente in modo **stocastico**) nel compito $y = f(x)$.

Un modello di Machine Learning $f_{\Theta}(x)$, parametrizzato da Θ , si dice che **risolve** il problema se esiste una scelta di parametri Θ tale che

$$f_{\Theta}(x) \approx f(x) \quad \forall x \in \mathbb{R}^d.$$

2.1.1 È apprendibile?

Un problema $y = f(x)$ può essere risolto da un algoritmo ML se e solo se **esiste** una relazione tra x e y . Ad esempio, non possiamo aspettarci di predire il meteo futuro in una particolare posizione usando informazioni sul prezzo azionario di una certa società. In quella situazione, ingresso e uscita sono chiaramente **indipendenti**, e non c'è alcuna possibilità di imparare qualcosa dall'uno usando l'altro.

Di conseguenza, il primo punto nella progettazione di un algoritmo ML è capire *se* esiste una correlazione tra ingresso e uscita del problema dato. Quando ciò accade, diciamo che il problema è **apprendibile**.

Il **Machine Learning** riguarda la comprensione di correlazioni (pattern).

2.1.2 È possibile raccogliarli?

Assumiamo che il problema $y = f(x)$ sia apprendibile. Dobbiamo capire se possiamo fisicamente raccogliere abbastanza dati x da poter comprendere la relazione tra essi e il corrispondente y .

Ad esempio, se vogliamo usare il ML per fare diagnosi tumorali sui pazienti, chiaramente il modo migliore per farlo sarebbe usare come ingresso i risultati clinici di ogni possibile esame medico sul paziente. Naturalmente, anche se questo funzionerebbe bene in pratica, non è possibile (e specialmente non è etico) sottoporre il paziente a migliaia di esami per una singola diagnosi.

Inoltre, per allenare un buon modello ML avremo bisogno di migliaia (a volte milioni) di punti dati, e non è sempre possibile scalare il problema in modo da riuscire a raccogliere dati sufficienti per risolverlo.

Raccogliere dati richiede **efficienza** e **scalabilità** del problema.

2.2 Raccogliere i dati

Raccogliere i dati è di solito il passo che richiede più sforzo nella progettazione di un algoritmo di Machine Learning. Infatti, anche ammesso che il nostro problema $y = f(x)$ sia risolvibile e che sia **teoricamente possibile** raccogliere dati sufficienti su di esso, non è detto che ciò sia **facile** in pratica.

In particolare, alcuni dati richiedono *tempo* per essere raccolti (è il caso tipico delle applicazioni biologiche o mediche), e raccogliere dati di buona qualità è difficile. Idealmente vorremmo usare un dataset pulito, in cui tutte le informazioni sono presenti, non ci sono valori mancanti (di solito indicati con NaN) e le informazioni non contengono rumore. Nella maggior parte dei casi questo è irrealistico, e sarà necessario sviluppare algoritmi per standardizzare e ripulire i dati. L'insieme di queste tecniche è chiamato **data cleaning** (pulizia dei dati).

2.2.1 Kaggle

Fortunatamente, per la maggior parte dei compiti a cui si può pensare, è possibile trovare dataset già pronti su internet. Ad esempio, siti come Kaggle e Google Datasets possono essere utili a questo scopo.

2.2.2 Dataset e array NumPy

I dataset in formato `.csv` sono ampiamente utilizzati, poiché sono facili da leggere e spesso hanno righe e colonne chiaramente etichettate. Tuttavia, da un punto di vista matematico e computazionale, questo formato non è ideale per il Machine Learning. In particolare:

- Lavorare direttamente con le stringhe è inefficiente.
- I nomi di righe e colonne non sono utili per gli algoritmi di apprendimento.
- La maggior parte degli algoritmi ML si aspetta dati sotto forma di vettori e matrici numeriche.

Per questo motivo, quasi sempre **convertiamo i dataset in matrici**, tipicamente rappresentate come array NumPy. Questa conversione avviene in due passi principali:

1. **Codifica dei valori categoriali (stringhe) in numeri.**
2. **Conversione del dataset in un array NumPy.**

Supponiamo di avere una feature categoriale, come il *meteo*, con possibili valori {soleggiato, piovoso, nuvoloso, ...}. Poiché il numero di valori possibili è limitato, possiamo assegnare ogni categoria a una classe.

Due strategie classiche di codifica sono:

- **Codifica a interi (Integer Encoding):** ogni classe C_k viene mappata a un indice intero k . Un difetto cruciale di questo metodo è che introduce un ordinamento artificiale tra le categorie. In Python si implementa con:

```
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
encoded = encoder.fit_transform(categories)
```

- **Codifica One-Hot (One-Hot Encoding):** ogni classe C_k viene mappata nel k -esimo vettore canonico $e_k \in \mathbb{R}^K$, che è tutto zero tranne un singolo 1 in posizione k . In Python si implementa con:

```
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder(sparse=False)
encoded = encoder.fit_transform(categories.reshape(-1, 1))
```

Dopo la codifica, il dataset può essere convertito in un array NumPy con:

```
import numpy as np
X = np.array(encoded_dataset)
```

Il risultato è una matrice:

$$X = [x_1, x_2, \dots, x_N] \in \mathbb{R}^{d \times N},$$

dove ogni colonna $x_i \in \mathbb{R}^d$ è un punto dati con d feature, e N è il numero di campioni. Le etichette corrispondenti sono raccolte in un vettore:

$$Y = [y_1, y_2, \dots, y_N]^T \in \mathbb{R}^N.$$

2.2.3 Strategie moderne di embedding

Sebbene la codifica a interi e la codifica one-hot siano semplici ed efficaci, non catturano le **relazioni semantiche** tra le categorie. Ad esempio, nella codifica one-hot, i vettori per *soleggiato* e *nuvoloso* sono completamente ortogonali, anche se sono più simili tra loro rispetto a *nevososo*.

Per affrontare questa limitazione, oggi si usano ampiamente gli **embedding appresi**:

- Nel **Natural Language Processing (NLP)**, parole o token vengono mappati in vettori densi in $\mathbb{R}^d$, chiamati **word embedding**. Questi vettori vengono appresi da grandi corpora di testo in modo che parole semanticamente correlate (ad es. *re* e *regina*) siano vicine nello spazio degli embedding.
- Nei moderni **Large Language Model (LLM)**, gli embedding sono usati a più livelli: per parole, sottoparole (subword) o token. Questi embedding vengono **appresi congiuntamente** ai parametri del modello, permettendo alla rete di scoprire automaticamente le relazioni durante il training.
- Strategie simili sono usate nei **sistemi di raccomandazione** (ad es. embedding di utenti e oggetti) e nel **graph learning** (embedding dei nodi).

In pratica, questo significa che invece di codificare manualmente le categorie, lasciamo che il modello **apprenda una rappresentazione** ottimale per il compito. In PyTorch, ad esempio, ciò si può fare con il layer `nn.Embedding`:

```
import torch
import torch.nn as nn

# Supponiamo di avere 100 possibili categorie, mappate in embedding a 16 dimensioni
embedding = nn.Embedding(num_embeddings=100, embedding_dim=16)

# Esempio: indice di categoria 5 -> vettore embedding in R^16
category_index = torch.tensor([5])
vector = embedding(category_index)
```

2.3 Progettazione (Design)

Progettare un modello di Machine Learning è uno dei passi più impegnativi, poiché richiede di scegliere il paradigma corretto in base al tipo di problema che vogliamo risolvere. In generale, gli algoritmi di apprendimento si possono raggruppare in tre categorie principali:

- **Apprendimento Supervisionato (Supervised Learning)**
- **Apprendimento Non Supervisionato (Unsupervised Learning)**
- **Apprendimento Auto-Supervisionato (Self-Supervised Learning)** (avanzato, non trattato in questo corso)

2.3.1 Apprendimento Supervisionato

Nell'**Apprendimento Supervisionato (SL)**, ci viene fornito un dataset composto da coppie *ingresso-uscita*:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N,$$

dove $x^{(i)} \in \mathbb{R}^d$ è l'ingresso (un vettore di feature) e $y^{(i)}$ è l'etichetta o il target corrispondente.

L'idea dell'apprendimento supervisionato è apprendere una funzione:

$$f_{\Theta} : \mathbb{R}^d \rightarrow \mathcal{Y},$$

parametrizzata da Θ , tale che $f_{\Theta}(x^{(i)}) \approx y^{(i)}$.

L'apprendimento supervisionato si può ulteriormente dividere in due compiti principali:

- **Regressione:** l'uscita $y^{(i)}$ è una quantità a valori reali, cioè $y^{(i)} \in \mathbb{R}^s$. Esempio: predire il prezzo di una casa a partire dalle sue caratteristiche.
- **Classificazione:** l'uscita $y^{(i)}$ appartiene a un insieme finito di classi, cioè $y^{(i)} \in \{1, \dots, K\}$. Esempio: classificare un'email come *spam* o *non spam*.

La qualità di un modello supervisionato è tipicamente misurata usando **funzioni di perdita (loss function)**, che quantificano la discrepanza tra il valore predetto $f_{\Theta}(x)$ e l'etichetta vera y . Durante il training, ottimizziamo Θ per minimizzare questa perdita sul dataset.

2.3.2 Apprendimento Non Supervisionato

Nell'**Apprendimento Non Supervisionato (UL)**, ci vengono forniti solo dati di ingresso, senza etichette:

$$\mathcal{D} = \{x^{(i)}\}_{i=1}^N, \quad x^{(i)} \in \mathbb{R}^d.$$

Qui, il compito non è predire un'uscita nota, ma piuttosto scoprire **strutture o pattern nascosti** nei dati. Il modello cerca di organizzare, comprimere o spiegare i dati in modo significativo.

Compiti comuni includono:

- **Clustering**: raggruppare i punti dati in cluster tali che i punti nello stesso gruppo siano più simili tra loro rispetto ai punti di altri gruppi. Esempio: segmentazione di mercato dei clienti in base al loro comportamento d'acquisto.
- **Riduzione della dimensionalità**: mappare dati ad alta dimensionalità in uno spazio a dimensionalità inferiore, preservando il più possibile l'informazione. Esempio: Analisi delle Componenti Principali (PCA) per la visualizzazione dei dati.

L'apprendimento non supervisionato è spesso usato quando i dati etichettati sono costosi o impossibili da ottenere, rendendolo uno strumento potente nell'analisi esplorativa dei dati.

2.3.3 Apprendimento Auto-Supervisionato

L'**Apprendimento Auto-Supervisionato (SSL)** è un paradigma emergente che si colloca tra apprendimento supervisionato e non supervisionato. A differenza dell'apprendimento supervisionato, non sono richieste etichette annotate da esseri umani. Il modello crea invece le proprie etichette risolvendo dei *pretext task* (compiti pretesto) definiti direttamente sui dati di ingresso.

Esempi di pretext task includono:

- Predire parole mancanti in una frase (usato in large language model come BERT).
- Predire parti mancanti di un'immagine o il fotogramma successivo di un video.
- Apprendimento contrastivo (*contrastive learning*), dove il modello apprende confrontando diverse viste dello stesso dato.

L'idea chiave è che, risolvendo questi compiti generati automaticamente, il modello apprende **rappresentazioni utili** dei dati, che possono poi essere raffinate (*fine-tuned*) su dataset etichettati più piccoli per compiti supervisionati.

Sebbene l'apprendimento auto-supervisionato sia diventato estremamente popolare nel Natural Language Processing e nella Computer Vision, in questo corso non lo tratteremo, poiché il nostro focus resterà sugli approcci classici **supervisionato** e **non supervisionato**.

In sintesi:

- **Apprendimento Supervisionato**: si apprende da dati etichettati (x, y) .
- **Apprendimento Non Supervisionato**: si apprende da dati non etichettati (x) .
- **Apprendimento Auto-Supervisionato**: si generano etichette a partire dai dati stessi per pre-allenare rappresentazioni.

Nelle prossime lezioni studieremo metodi di apprendimento supervisionato (sia regressione che classificazione) e metodi di apprendimento non supervisionato (come clustering e riduzione della dimensionalità). L'apprendimento auto-supervisionato, per quanto potente, verrà lasciato a corsi più avanzati.

2.4 Allenamento (Training)

La **fase di training** è il cuore di ogni algoritmo di Machine Learning. Una volta definito un modello e raccolti i dati, il training è il processo attraverso cui il modello *apprende* i migliori parametri Θ per approssimare la funzione sottostante che stiamo cercando di catturare.

Formalmente, ricordiamo che un modello di Machine Learning può essere scritto come una funzione parametrizzata:

$$f_{\Theta} : \mathbb{R}^d \rightarrow \mathcal{Y},$$

dove Θ rappresenta i parametri (pesi, bias, coefficienti, ecc.) che determinano come il modello elabora gli ingressi.

2.4.1 Il ruolo della funzione di perdita

Il training richiede un modo per misurare **quanto siano buone** le predizioni di un modello. Questo si fa tramite una **funzione di perdita** (*loss function*, chiamata anche funzione di costo o funzione obiettivo), indicata come:

$$\mathcal{L}(\Theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\Theta}(x^{(i)}), y^{(i)}),$$

dove:

- $(x^{(i)}, y^{(i)})$ sono i punti dati del training set,
- $f_{\Theta}(x^{(i)})$ è la predizione del modello,
- $\ell(\cdot, \cdot)$ è una misura d'errore locale (ad es. errore quadratico, cross-entropy).

La funzione di perdita quantifica la discrepanza tra predizioni e uscite vere. Il **problema del training** è quindi il problema di ottimizzazione:

$$\Theta^* = \arg \min_{\Theta} \mathcal{L}(\Theta).$$

2.4.2 Il training come problema di ottimizzazione

Da un punto di vista matematico, allenare un modello di Machine Learning equivale a risolvere un **problema di ottimizzazione**. È qui che entrano in gioco tecniche di ottimizzazione come la **Discesa del Gradiente**.

- In regressione, una scelta comune di loss è l'**Errore Quadratico Medio (MSE)**:

$$\ell(f_{\Theta}(x), y) = \|f_{\Theta}(x) - y\|^2.$$

- In classificazione, una scelta comune è la **Cross-Entropy Loss**:

$$\ell(f_{\Theta}(x), y) = - \sum_{k=1}^K \mathbf{1}_{\{y=k\}} \log p_{\Theta}(y = k \mid x),$$

dove $p_{\Theta}(y = k \mid x)$ è la probabilità predetta dal modello per la classe k .

Indipendentemente dalla scelta della loss, allenare significa trovare valori dei parametri Θ che minimizzano $\mathcal{L}(\Theta)$.

2.4.3 Perché la Discesa del Gradiente?

Nella maggior parte delle applicazioni pratiche di Machine Learning, la funzione di perdita $\mathcal{L}(\Theta)$ **non è risolvibile analiticamente**:

- È solitamente non lineare nei parametri.
- Può coinvolgere milioni (o addirittura miliardi) di parametri.
- La dimensione del dataset N può essere estremamente grande.

Per questo motivo, soluzioni in forma chiusa (come nella semplice regressione lineare) in generale non sono possibili. Ci affidiamo invece ad **algoritmi di ottimizzazione iterativi**. Il più fondamentale è la **Discesa del Gradiente (GD)**.

L'idea chiave della Discesa del Gradiente è semplice:

- Calcolare il gradiente della funzione di perdita rispetto ai parametri, $\nabla_{\Theta} \mathcal{L}(\Theta)$.
- Aggiornare i parametri nella direzione che diminuisce la loss:

$$\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}(\Theta),$$

dove $\eta > 0$ è il **learning rate** (tasso di apprendimento).

Questa regola di aggiornamento iterativa sposta gradualmente i parametri verso valori che minimizzano la loss.

2.4.4 L'impostazione stocastica

Quando i dataset sono molto grandi, calcolare il gradiente esatto $\nabla_{\Theta}\mathcal{L}(\Theta)$ su tutti gli N campioni può essere proibitivamente costoso. Per affrontare questo problema, si usa la **Discesa del Gradiente Stocastica (SGD)**, dove il gradiente viene stimato usando solo un sottoinsieme (un *mini-batch*) dei dati:

$$\nabla_{\Theta}\mathcal{L}(\Theta) \approx \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\Theta} \ell(f_{\Theta}(x^{(i)}), y^{(i)}),$$

con $\mathcal{B} \subset \{1, \dots, N\}$.

Questo riduce drasticamente il calcolo per ogni passo e introduce rumore negli aggiornamenti, il che, paradossalmente, spesso aiuta a sfuggire da minimi locali poco buoni.

2.4.5 Il training come motivazione per l'ottimizzazione

L'importanza del training nel Machine Learning è duplice:

1. **Concettuale:** il training è il momento in cui un modello passa da uno stato casuale, privo di informazione, a uno che codifica conoscenza utile sui dati.
2. **Matematica:** il training trasforma il problema dell'apprendimento in un problema di ottimizzazione.

Questo motiva perché l'ottimizzazione, e in particolare la **Discesa del Gradiente e le sue varianti**, sia centrale nel Machine Learning. Nelle prossime due lezioni studieremo in dettaglio la Discesa del Gradiente e la Discesa del Gradiente Stocastica, analizzandone sia i fondamenti matematici che le implementazioni pratiche.

2.5 Calibrazione (Tuning)

La maggior parte dei modelli di Machine Learning ha diversi **iperparametri** che non vengono appresi direttamente durante il training, ma devono essere impostati dal practitioner. Esempi includono il learning rate nella Discesa del Gradiente, il numero di layer in una rete neurale, o il numero di cluster in un algoritmo di clustering.

Il tuning consiste nell'aggiustare sistematicamente questi iperparametri per migliorare le prestazioni del modello. Questo si fa di solito tramite un processo per **tentativi ed errori** (*trial-and-error*), guidato dalla conoscenza pregressa del modello e valutando le prestazioni su un validation set separato.

L'equilibrio è delicato: un tuning scadente può portare a **underfitting** (il modello è troppo semplice per catturare la struttura dei dati) oppure a **overfitting** (il modello memorizza il training set senza generalizzare).

2.6 Test

Una volta che un modello è stato allenato (parametri ottimizzati) e calibrato (iperparametri scelti), dobbiamo valutare la sua **capacità di generalizzare**. Questo significa valutare quanto bene il modello si comporta su dati che non ha mai visto.

Il test è quindi un passo fondamentale: ci permette di verificare se il modello ha davvero appreso pattern utili, oppure se ha semplicemente memorizzato i dati di training.

2.6.1 Dati di training vs dati di test

Usare lo stesso dataset sia per il training che per il test è fuorviante. Sui dati di training il modello ha già osservato le uscite vere, quindi potrebbe semplicemente **memorizzarle**. Questo fenomeno è noto come **overfitting**.

Per evitarlo, il dataset viene tipicamente suddiviso in:

- **Training set:** usato per aggiustare i parametri del modello,
- **Validation set:** a volte usato per calibrare gli iperparametri,

- **Test set:** usato una sola volta, alla fine, per fornire una stima non distorta delle prestazioni. Formalmente, se partiamo da N punti dati, selezioniamo:

$$N_{\text{train}} < N, \quad N_{\text{test}} = N - N_{\text{train}},$$

con il test set tenuto da parte durante l'intero processo di training e tuning.

2.6.2 Metriche di valutazione

La scelta della metrica dipende dal compito:

- **Regressione:**

- Errore Quadratico Medio (MSE):

$$\text{MSE} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} (f_{\Theta}(x^{(i)}) - y^{(i)})^2$$

- Errore Assoluto Medio (MAE), punteggio R^2 , ecc.

- **Classificazione:**

- Accuratezza (*accuracy*): la proporzione di campioni classificati correttamente,
- Precision, Recall, F1-score (specialmente su dataset sbilanciati),
- AUROC (Area Under the ROC Curve) per classificatori probabilistici.

L'idea chiave è valutare non solo quanto le predizioni siano vicine alle etichette, ma anche se il modello stia facendo il **tipo giusto di errori** per il problema in questione.

2.6.3 Underfitting vs Overfitting

Un aspetto cruciale del test è comprendere il **comportamento di generalizzazione** di un modello. Possono verificarsi due situazioni estreme:

- **Underfitting:** il modello è *troppo semplice*. Non riesce a catturare la struttura sottostante dei dati, portando a un **errore alto** sia sul training set che sul test set. Esempio: cercare di adattare una retta a dati fortemente non lineari.
- **Overfitting:** il modello è *troppo complesso*. Si adatta troppo strettamente ai dati di training, catturando anche il rumore e i pattern irrilevanti. Questo porta a un **errore basso** sul training set ma a un **errore alto** sul test set. Esempio: un albero decisionale profondo che classifica perfettamente il training set ma fallisce su dati nuovi.

Il modello ideale si trova **tra underfitting e overfitting**, raggiungendo un basso errore di training e un errore di test comparabile. La differenza tra questi due errori è chiamata **generalization gap** (divario di generalizzazione). Un divario piccolo indica che il modello generalizza bene.

2.6.4 Riepilogo

- Il **tuning** aggiusta gli iperparametri per controllare la complessità del modello.
- Il **test** valuta la generalizzazione su dati non visti.
- **Underfitting:** il modello è troppo semplice, fallisce ovunque.
- **Overfitting:** il modello è troppo complesso, memorizza il training set ma fallisce su campioni nuovi.
- L'obiettivo finale è minimizzare il **generalization gap** e garantire che il modello apprenda pattern che si trasferiscono oltre il training set.

*Traduzione italiana del materiale del corso “Statistical and Mathematical Methods for Machine Learning”
(pagina originale in inglese).
Materiale originale di Davide Evangelista — © 2023.*